

Dhaga Workbench

Reading what customers tell Dhaga & Co., at scale: a checked support reply in about 10 seconds, and a real reason for every "Other" return.

Live app: nagarjuna2026-dhaga-mvp.hf.space · Repository branch: feat/base-backed-wismo-mvp · 3 October 2026

The project in one paragraph

Dhaga's customers keep telling the company what's wrong, in tickets and in return comments, often in Hinglish. Nobody can read it all, so replies take hours and most returns go unexplained. We built one approach for both problems: **AI reads the words, code checks the facts against Dhaga's own data, and a person makes the final call.** It runs live on Supabase data, uses Claude through LangChain, and costs under ₹9,000 a week in AI calls at Dhaga's volume.

≈ 10 s

for a checked draft reply (first response today: 9 hours)

70 / 70

test tickets understood correctly; 0 risky tickets auto-drafted

216 / 216

"Other" returns now carry a real reason

215 / 216

return reasons correct; the one miss went to a person

< ₹9,000

AI cost a week at Dhaga's volume, both problems together

53

automated tests passing, run at no cost

1 · MAIN GOALS

What the project set out to do

GOAL	WHAT IT MEANS IN PRACTICE
Answer routine support tickets faster	Give agents a checked draft for "where is my order" and similar tickets, so they review instead of typing. Today's first response is 9 hours.
Explain every "Other" return	Turn free-text return comments into a real reason (for example "fit, too tight at the shoulders"), so the category team knows which products to fix.
Never let AI act alone on customers	Facts come from Dhaga's data through code. A person approves every customer reply and decides every uncertain return reason.
Fail visibly	When the system can't answer (lost parcel, unclear message, AI down), it says so on screen and hands the case to a person.
Be cheap and easy to run	Pay per use with no model training; one container; plain Python rules; no ML engineer needed (Dhaga has none).

2 · THE PROBLEM

Customers tell Dhaga what's wrong. Nobody can read it all.

Support

"Fifty-eight percent of tickets are some version of where is my order. My agents copy and paste the same four replies all day. Average first response is nine hours."

Arpita, Head of CX

- About **9,000 tickets a week** on Freshdesk and WhatsApp, handled by 34 agents.
- **58%** ask "where is my order?", about **5,220 a week** (our calculation).
- Agents look up each order and paste a canned reply; first response averages **9 hours**.

Returns

"Returns are thirty-one percent overall. When I read the Other box by hand, most of it is about fit, but I can only read a few hundred at a time."

Neha, Category Head

- **31%** of orders come back; **44%** of returns say only "Other" plus a typed comment.
- That is about **6,550 unexplained returns a week** ($48,000 \times 31\% \times 44\%$, our calculation).
- Without a reason, Dhaga can't fix the product, size chart or photo.

In one line: customers explain their problems in their own words, often Hinglish, and nothing turns those words into something a team can act on. Every number above is from the client brief, except those marked as our calculation.

3 · HOW WE FOUND IT

We ranked six problems before writing code

We read the brief first: what each person in the company complains about, and the numbers behind it. We listed six possible problems and scored each on five questions: does a named person own it, is the pain measured, does the data exist, can we show it working end to end, and what happens if we get it wrong.

#	PROBLEM · OWNER	PAIN	DATA READY	DEMOABLE	RISK IF WRONG
1	Support swamped by "where is my order" · Arpita	High	High	High	Low
2	Returns with no real reason · Neha	High	Medium	Medium	Low
3	COD orders returned undelivered · Faizan	Highest (≈ ₹9.1 lakh/week)	Low	Low	High
4	Slow, inconsistent product copy · Vivek	Medium	High	High	Medium
5	410,000 reviews nobody reads · no owner	Low	High	High	Low
6	Two-week analyst queue · Karthik	Medium	High	Low	High

Why these two, and why not the biggest money first

- **#1 and #2 are the same fix for two owners:** read the customer's words, turn them into a reason, let a person decide. One system serves both.
- **COD return-to-origin** costs the most: about $48,000 \times 61\% \text{ COD} \times 26\% \text{ failed} \approx 7,600 \text{ deliveries} \times ₹120 \approx ₹9.1 \text{ lakh a week}$ (our calculation). It ranks third because it needs customer and address history we don't have, sends messages to customers before dispatch, and can't be shown reducing failures within a short project. It is first in line for the next engagement.
- **Reviews** have no owner today, so they are an idea rather than a problem. They become useful as an input to returns and product copy (phase 2).

The full write-up, with the seven items the brief requires, is in `docs/discovery-note.md`.

One approach for both: AI reads, code checks, a person decides

Support: CX Copilot

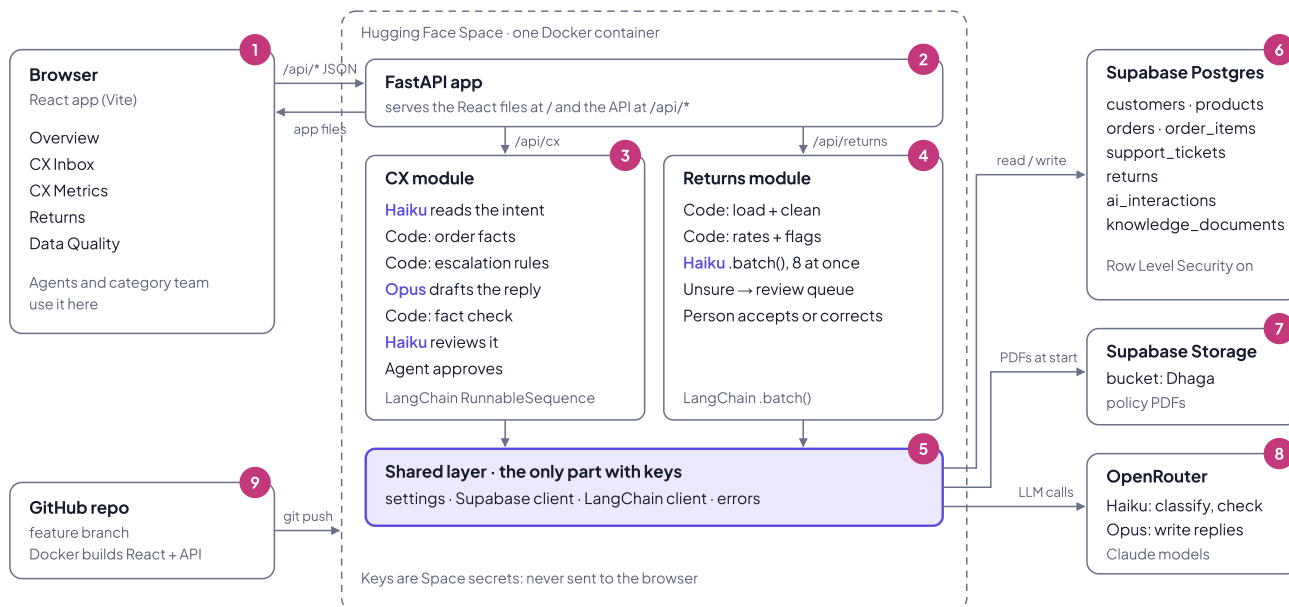
1. **AI** Reads the message and works out what the customer wants.
2. **Code** Finds the order and works out the facts: status, how late, cancellable or not.
3. **Code** Risky cases (lost parcel, damage, return-to-origin, unclear) go straight to a person.
4. **AI** For the rest, writes a reply using only the real facts and Dhaga's policy.
5. **Code** + **AI** Checks every date, amount and order number; rewrites once if anything is wrong.
6. **Human** The agent approves, edits or rejects. Nothing reaches a customer without them.

Returns: Returns Insights

1. **Code** Loads and cleans every return; rejects broken rows with a reason.
2. **Code** Works out which products, sizes and colours come back too often.
3. **AI** Reads every "Other" comment and gives it a reason, a detail and a body area.
4. **Code** Anything unsure (below 70% confidence, or unclear) goes to a review queue.
5. **Human** Neha's team accepts or corrects; their answer always wins.
6. Dhaga can now see **why** each problem product comes back.

One web app, one backend, one database

The browser only ever talks to the backend. The backend is the only part that holds the keys to the database and to Claude. The numbered parts are explained below the diagram.



Purple text marks the steps where Claude is called; everything else is plain code or a person. Pink numbers match the list below.

#	PART	WHAT IT DOES
1	Browser (React + Vite)	The app agents and the category team use: Overview, CX Inbox, CX Metrics, Returns, Data Quality. It calls <code>/api/*</code> with JSON and never sees a key.
2	FastAPI app	One Python web server. It serves the built React files at <code>/</code> and the API at <code>/api/cx/*</code> and <code>/api/returns/*</code> , with one error format for every endpoint and a health check at <code>/api/health</code> .
3	CX module	The support workflow, built as a LangChain RunnableSequence (section 7). Reads tickets and orders, runs the steps in section 8, saves every run, records the agent's decision.
4	Returns module	Loads and cleans returns, counts return rates, flags problem products, sends "Other" comments to Claude with LangChain's <code>.batch()</code> , and manages the review queue.
5	Shared layer	Settings, the Supabase client and the LangChain client. The only code that holds keys. The two modules never call each other; both go through here, so they could later become separate services.
6	Supabase Postgres	Dhaga's data: customers, products, orders, order items, 1,800 support tickets, 563 returns, plus <code>ai_interactions</code> (every AI run and decision). Row Level Security is on, so the public browser key can't read customer data.
7	Supabase Storage	The bucket for policy PDFs (delivery, cancellation, returns, COD, escalation), loaded once at start. Placeholder texts are used until the real PDFs are uploaded.

8	OpenRouter → Claude	Claude Haiku 4.5 (classify, check, read return comments) and Claude Opus 5.5 (write customer replies), reached through LangChain.
9	GitHub → Hugging Face	The repository branch is pushed to the Hugging Face Space, which builds one Docker image (React build, then Python) and runs it. Keys are Space secrets.

6 · DATA INGESTION

How data gets in, gets cleaned, and gets saved

Where the data comes from

TABLE (SUPABASE)	ROWS	USED FOR
customers , products	300 / 100	Names for replies; product names, categories and SKUs
orders , order_items	2,500 / 2,500	Order status, courier, tracking, expected date, amount; size and colour per item
support_tickets	1,800	The CX inbox: message, channel, status, linked order
returns	563	Return reason, customer comment, and the saved AI and human reasons
ai_interactions	grows	Every Copilot run, every classify run, every agent decision, with cost and time
knowledge_documents + Storage	-	Which policy PDFs to load for each kind of ticket

Support data, step by step

1. **Read with joins in one request.** `SupabaseRepository` (`backend/modules/cx/repository.py`) asks Supabase for a ticket together with its customer, or an order together with its items and product names, in one query.
2. **Translate statuses.** Supabase uses `OPEN / IN_PROGRESS / CLOSED` ; the app uses Open, Drafted, Resolved and Escalated. The repository maps between them in both directions.
3. **Turn rows into facts with code.** `build_facts()` works out days late, whether the order can still be cancelled, and whether it is inside the 7-day return window. Models never do this arithmetic.
4. **Save every run.** Each Copilot result, its cost, its models and prompt versions go to `ai_interactions` ; the agent's decision updates that row and the ticket's status.

Returns data, step by step

1. **Load.** `supabase_loader.py` reads products, order items (with their orders) and returns (with their item, size, colour and product), and reshapes them to the same columns as the original demo files. The rest of the pipeline doesn't care where data came from.
2. **Validate.** `validation.py` rejects broken rows with exactly one reason each: blank fields, duplicates, bad dates, unknown products, a return pointing to a missing order, or a product that doesn't match its order. On the live data, 0 of 563 were rejected.

3. **Normalise.** `normalization.py` maps size spellings (medium, M, " m ") and colour names (gulabi, pinkish → Pink), and maps dropdown reasons ("Size/Fit" → Fit). "Other" with a comment is marked as waiting for the AI.
4. **Apply saved reasons.** AI reasons and a person's corrections, stored on each return, replace "Unclassified". A person's answer always wins.
5. **Count and flag.** Return rates per product, size, colour and category; a segment is flagged at 1.5 times the shop's return rate with at least 8 returns (12 products on the live data).

Data problems we found and handled: the synthetic tickets had only 20 distinct messages across 1,800 rows, and the 216 "Other" returns only 6 distinct comments, so we regenerated them with realistic Hinglish and English text and a known correct answer for each (the originals are backed up and can be restored). 290 of 563 returns belong to orders that were never delivered; we keep them and show a data-quality warning. Two products share one name, so the app shows SKUs.

7 · LANGCHAIN

Where the framework does the work

LangChain is used in three places. Business rules and lookups stay in plain Python on purpose, so Dhaga's engineers can read and change them.

1. Every model call is a LangChain chain

```
PROMPT = ChatPromptTemplate.from_messages([("system", "{system}"), ("human", "{user}")])
chain = PROMPT | ChatOpenAI(model="anthropic/claude-haiku-4.5", temperature=0)
    .with_structured_output(TicketClassification, method="json_schema", strict=True)
result = chain.invoke({"system": prompt_file_text, "user": customer_message})
```

- **ChatPromptTemplate** builds the messages from a versioned prompt file and the customer's text (a "{" in a message can't break it).
- **ChatOpenAI** calls Claude through OpenRouter, with the model, temperature, token limit and retries.
- **with_structured_output** sends a strict JSON schema and turns the answer into a checked Pydantic object. If the answer doesn't fit the schema, the case goes to a person.

2. The CX workflow is a RunnableSequence

```
COPILOT_CHAIN =
    assign(classification = classify_intent)           # AI: Haiku 4.5
  | assign(order_info     = find_order)               # code
  | assign(facts          = order_facts)              # code
  | assign(decision       = apply_rules)              # code
  | assign(policy         = pick_policy)              # code
  | assign(outcome = RunnableBranch(                 # routing
      (needs a person,    hand_to_person),
      (no order found,    ask_for_order_number),
      draft_and_check))
  | build_and_log_result                             # AI + code, one redraft
                                                    # code: save to ai_interactions
```

Each step adds to a shared state. The **RunnableBranch** is the routing pattern. The redraft loop lives inside `draft_and_check`, because a sequence only flows forward.

3. Returns comments run with .batch()

```
outputs = chain.batch(all_comment_prompts, config={"max_concurrency": 8},
    return_exceptions=True)
```

One chain, every "Other" comment, 8 at a time: 216 comments in 50 seconds instead of about 6 minutes. A failed comment comes back as an error for that item only; the rest carry on.

Models and settings

STEP	MODEL	SETTING	WHY
Classify a ticket	Claude Haiku 4.5	temperature 0	High volume, must give the same answer every time
Write the reply	Claude Opus 5.5	reasoning effort "low"	Customer-facing writing; Opus 5.5 doesn't accept a temperature, so effort controls it
Check the reply	Claude Haiku 4.5	temperature 0	A yes/no judgement after code has checked the facts
Read return comments	Claude Haiku 4.5	temperature 0	Mapping messy text to a fixed list of reasons

8 · RESOLVING PROCEDURE

What happens to a ticket, step by step

A real example from the live data: ticket **TKT200056**, *"Excuse me shirt fit nahi hua, return karna hai (order no DHC101251) very bad service"*.

- 1
- The agent clicks Analyse.** The browser sends `POST /api/cx/tickets/TKT200056/analyze`. The ticket and customer are read from Supabase.

- 2 **Read the intent** Haiku 4.5 . Result: **Return / refund**, Hinglish, order number **DHC101251** taken from the message.

- 3 **Find the order** Code . The number in the message wins; the code checks the order belongs to this customer.

- 4 **Work out the facts** Code . Delivered on 24 August, 40 days ago, so **outside the 7-day return window**.

- 5 **Apply the rules** Code . Not risky, so drafting may go ahead. (A lost parcel, damage, return-to-origin, an unclear message or someone else's order would stop here and go to a person.)

- 6 **Pick the policy** Code . Return or refund → the Returns policy.

- 7 **Write the reply** Opus 5.5 , in Hinglish, from the facts and the policy only.

- 8 **Check it** Code + Haiku 4.5 . Every date, amount and order number must match the facts, and Haiku checks it answers the question and follows the policy. Here the **first draft failed the check, so Opus redrafted once**, and the second draft passed.

- 9 **Save and show**. The run is saved to ai_interactions ; the ticket becomes "Drafted". Total: 2 drafts, about \$0.027.

- 10 **The agent decides** Human . Approve, edit or reject. The ticket becomes Resolved or Escalated.

The reply the agent saw: "Hi Priya, aapko pareshani hui iske liye sorry. Aapka order DHC101251 24 August ko deliver hua tha, yaani 40 din ho gaye hain. Hamari policy ke hisaab se return sirf delivery ke 7 din ke andar ho sakta hai... Team Dhaga"

What happens to an "Other" return

- 1 **The category team clicks "Classify with AI"**. The pipeline picks every "Other" comment still waiting.

- 2 **Haiku reads each comment** AI , 8 at a time. Example: *"shoulders pe bahut tight hai"* → **Fit · too tight · shoulders**.

- 3 **Save**. Reason, detail and confidence are written to that return in Supabase.

- 4 **Unsure ones go to review** Code . Below 70% confidence, or "unclear" (for example "nahi" or "ok very disappointed"). 18 of 216 went there.

- 5 **A person accepts or corrects** Human . Charts and flagged products update with the new reasons.

When something goes wrong

SITUATION	WHAT HAPPENS
Lost parcel, damaged or wrong item, return-to-origin	Straight to a Tier 2 agent with the reason shown; no draft
Unclear message, or someone else's order number	Goes to a person; no order data shown
Draft invents a date or amount	Caught by code, redrafted once, otherwise a person
Claude down or out of credit	Standard template replies with a "Fallback mode" banner
Supabase down, or a table missing	The status bar turns red, or a "Setup needed" warning names the SQL file

9 · RESULTS

Achievement statistics

MEASURE	RESULT	HOW IT WAS MEASURED
Time to a checked support draft	≈ 10 seconds	Timed on real Supabase tickets (first response today: 9 hours)
Support intent accuracy	70 / 70	10 labelled tickets per intent, Haiku 4.5, prompt version 2
Risky tickets auto-drafted	0	Same test: no lost-parcel, damage or unclear ticket got a draft
Keyword rules without a model	72.7%, 99 unsafe	All 1,800 tickets: why the model earns its place
"Other" returns explained	216 / 216	Live run on Supabase; 18 sent to a person for review
Return reason accuracy	215 / 216	Against the known reason for each comment; the one miss went to review
Fit detail (body area)	105 / 105	Shoulders, waist, bust, sleeves or length correct for every fit comment
Return comments, speed	50 s for 216	LangChain .batch(), 8 at a time (about 6 minutes one by one)
Problem products flagged	12	≥ 1.5 times the shop return rate (22.5%) with ≥ 8 returns
Automated tests	53 passing	Run with fake models and demo data: free and repeatable

The test text is generated from templates, so it is easier than real customer messages. 0 errors in 70 means the real error rate is probably below about 4%, not zero. Before going live we would score 200 real tickets labelled by Dhaga's team.

Under ₹9,000 a week of AI at Dhaga's volume

WORK	WEEKLY VOLUME	COST EACH (MEASURED)	PER WEEK
Support replies	9,000 tickets (brief)	\$0.0104 ≈ ₹0.89	≈ ₹8,050 (upper bound)
Returns comments	≈ 6,550 (our calculation)	\$0.00105 ≈ ₹0.09	≈ ₹590
Total AI cost			≈ ₹8,640

9,000 × \$0.0104 = \$93.60 ≈ ₹8,050

6,550 × \$0.00105 = \$6.88 ≈ ₹590

(₹86 per \$ is our assumption)

What's included: AI model calls only, measured on real Supabase data. Support is an upper bound: risky tickets skip the expensive draft and cost about ₹0.08.

Not included: hosting and people's time. AI doesn't replace agents; it turns minutes of typing into a one-minute review. Agent cost per hour and cost per return aren't in the brief; they are our first questions.

How we know it works

- **53 automated tests** cover facts and rules, the fact check, the fallback, the full Copilot workflow (normal path, redraft, two failed checks, someone else's order, refusal, outage), the API, the LangChain client and batch, the returns classifier and the review endpoints.
- **Two accuracy scripts:** `evals/intent_eval.py` (support, about \$0.07 a run) and `evals/returns_eval.py` (returns, free, reads saved results).
- **The brief's ground rules:** two models with stated settings, structured output everywhere, code for facts and models for language, visible failures, a person reviewing every customer reply.
- **Things that broke and were fixed:** the deploy reported success while the app was down (a missing Space header); the live database used different status names and lacked two tables; Row Level Security was off; measuring cost revealed that damaged-item tickets got automatic drafts, now a Tier 2 rule (10 / 10 in the test).

Fix the cause, then the biggest money

NEXT STEP	WHAT IT INVOLVES
Fix the cause of returns	Four of six return reasons start on the product page: fit, colour, fabric and unanswered questions. Read the 410,000 unread reviews, then write fit notes ("runs small, size up") and fact-checked copy. An 11-week plan with a 50-product pilot against 50 unchanged products before rolling out. Estimated ≈ \$205 once to tag all reviews and ≈ \$1.80 a week for new listing copy.
COD return-to-origin	The biggest money problem (≈ ₹9.1 lakh a week), with customer and address history from Faizan's team.
Test on real data	Score 200 real Freshdesk tickets and 200 real "Other" comments labelled by Dhaga's team before going live.

13 · ASSUMPTIONS AND LIMITS

What we don't know yet

- **Biggest assumption:** the 9-hour first response is mostly agent handling time, not tickets waiting unopened. One week of Freshdesk timestamps would show whether that's wrong; if it is, staffing and routing matter more than drafting.
- The data is synthetic, and the regenerated test text is easier than real customer text.
- Policy texts are placeholders until the real PDFs are uploaded; the demo has no login.

Short version to say out loud: "Dhaga's customers keep telling them what's wrong, and nobody can read it all. We built one approach that reads it, checks it against real data, and lets a person decide. It drafts support replies in about 10 seconds instead of a 9-hour first response, and gives every unexplained return a real reason, for under ₹9,000 a week of AI."